

Hole-Driven Interactive Assembly for LLM comprehension

Goal: A framework for agentic AI to understand binary files (assembly)

Problem

LLM struggles to understand binary files. Previous work [1] has shown that feeding assembly into LLM directly leads to context explosion, which further increases the rate of hallucination. The authors also note that a whole binary cannot be efficiently put into the context window due to limitation of tokens. Therefore, the authors provide a pre-processing engine to convert the binary into “data” which can be queried (through the use of MCP and AI Agent). Their pipeline has two major limitation:

1. Relying on a well-established decompiler

Pipelines for binary often include the use of a decompiler, notably IDA/Ghidra/BinaryNinja. These tools, during the AI Agent trend, support MCP servers. The LLM must (fully) trust this decompiled code produced by these tools.

2. Targeting a specific part of the binary

The binary program can be magnitude of tokens more than the original sourcecode. Therefore, we cannot feed LLM the whole program, and accept contextual losses when working with large binary files.

Solution

The proposed approach is an editable-intermediate-language supporting the AI to “interact” and “fix” the lifted [decompiled] program. Designing this language must align with the LLM philosophy. The idea is to prevent too much context being fed to the LLM, as well as preventing the “rewrite everything” approach made by LLM when a fix is needed. The idea is highly inspired by two concepts, [structural editing](#) [2], [3], [4] and [webs](#) [5].

Structural Editing

Structural editing technique allows an edit to maintain the program’s consistency, usually through types. An edit will be typed, and that changes the current state of the program accordingly to the rules.

bla bla bla, i’m still reading the idea

With a structural editor, the LLM engine cannot make arbitrary changes. Rather, changes must matches the current acceptable state of the program. For example, if it wants to change a variable into a different struct, the edit is accepted if the struct accesses are correct accordingly.

The example below shows an object that has its name the first element. The LLM can only change this object’s type to struct having a char* as the first element. This works because print_name uses the first element as a char.

```
var obj: struct { char* name; ...attr };  
print_name(obj);
```

Webs

This is a new idea in PL theory, but easy to understand. Every variable/function of a program has a producer (where you create it) and consumers (where you use it). Link them together into a web (graph/tree whatever), thus a program can be represented as a *network of webs*. A transformer works over a web, propagates the changes across the web. The paper only describe basic transformations, but we can think of something good.

For example, the program below can change its type accordingly if we make a transformation from `int` to `bool`.

```
-void f(x int) int {  
+void f(x bool) bool {  
    return x;  
}  
  
void main() {  
-    f(1) + f(2);  
+    f(true) && f(false);  
}
```

Designing Intermediate Language

Probably a variant of typed assembly [6]. But we have to adapt it so that it support the structural editor manner and webs.

Pipeline

A simple framework will lift the assembly into the typed assembly, with holes and some web connections. Then we expose a MCP server for the AI Agent to work with this editor. The AI Agent can query the code (preferably by web ids), and perform some analysis to refine the program. Refinement are edits which are typed and checked before applying.

Bibliography

- [1] H. A. Hung, N. A. Khoa, L. K. M. Dang, and N. A. Khuong, “AI AGENT FOR BINARY PATCH DIFF: LOCALIZING EXPLOITABLE VULNERABILITIES IN CLOSED-SOURCE BINARIES,” 2026.
- [2] C. Omar, I. Voysey, M. Hilton, J. Aldrich, and M. A. Hammer, “Hazelnut: a bidirectionally typed structure editor calculus,” *ACM SIGPLAN Notices*, vol. 52, no. 1, pp. 86–99, 2017.
- [3] J. Prinz, H. Blanchette, and L. Lampropoulos, “Pantograph: A Fluid and Typed Structure Editor,” *Proceedings of the ACM on Programming Languages*, vol. 9, no. POPL, pp. 802–831, 2025.
- [4] M. D. Adams, E. Griffis, T. J. Porter, S. V. Satish, E. Zhao, and C. Omar, “Grove: A Bidirectionally Typed Collaborative Structure Editor Calculus,” *Proceedings of the ACM on Programming Languages*, vol. 9, no. POPL, pp. 2176–2204, 2025.
- [5] B. Quiring, D. Van Horn, J. Reppy, and O. Shivers, “Webs and Flow-Directed Well-Typedness Preserving Program Transformations,” *Proceedings of the ACM on Programming Languages*, vol. 9, no. PLDI, pp. 748–772, 2025.
- [6] G. Morrisett, K. Crary, N. Glew, and D. Walker, “Stack-based typed assembly language,” *Journal of Functional Programming*, vol. 12, no. 1, pp. 43–88, 2002.